# SIMATS ENGINEERING

# CSA6303- Threat Intelligence and Network Security

# A3-Malware Analysis Assignment

Assessment Questions

| | | | |
|---|---|---|---|
| **Subject** | Threat Intelligence | **Total Marks** | 30 |
| **Questions** | 3 (all compulsory) | **Marks per Question** | 10 |
| **REg.No** | 192465027 | **Name** | RESHMA S |
| **Faculty** | Dr.V.Parthipan | **Slot** | A |

*Instructions: Answer all questions. Each question carries 5 marks. Support answers with examples and diagrams where appropriate.*

***CO3:*** Perform threat analysis using intelligence models and utilize industry tools for monitoring and detection.

***Code of Conduct:***

*I RESHMA S [192465027] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

## Signature of Student : RESHMA S

## Questions :

### Exercise 1: Threat Intelligence Platform Investigation

Objective: To investigate suspicious IP addresses, domains, and file hashes using a threat intelligence platform.

Practical Tasks:

- Examine sample IoCs using VirusTotal or another authorized threat intelligence platform.
- Review available reputation scores and threat classifications.
- Record the findings and recommend appropriate security actions.

Expected Output: A threat intelligence investigation report.

### Exercise 2: Security Log Collection and Correlation

Objective: To collect and correlate security events from different sources.

Practical Tasks:

- Collect sample authentication, firewall, and web server logs.
- Identify common IP addresses, timestamps, and user activities.
- Correlate events to identify suspicious patterns.

Expected Output: A correlated security event summary.

### Exercise 3: Threat Hunting Using Network Logs

Objective: To identify suspicious network activities through proactive threat hunting.

Practical Tasks:

- Examine sample network traffic logs.
- Identify unusual outbound connections and repeated connection attempts.
- Investigate suspicious communication patterns and document relevant IoCs.

Expected Output: A threat-hunting report with findings and recommendations.

**Exercise 1: Threat Intelligence Platform Investigation**

This script queries the VirusTotal API v3 to investigate IoCs (IP addresses, domains, and file hashes) and generates a Threat Intelligence Investigation Report.

Python

```
import json

import requests

# Replace with your VirusTotal API key

API_KEY = "YOUR_VIRUSTOTAL_API_KEY"

HEADERS = {"x-apikey": API_KEY}

BASE_URL = "https://www.virustotal.com/api/v3"

# Sample IoCs to investigate

IOCS = {

    "ips": ["8.8.8.8", "185.220.101.5"],  # Clean IP vs Tor exit node

    "domains": ["google.com", "malicious-phishing-site.top"],

    "hashes": ["44d88612fea8a8f36de82e1278abb02f"],  # EICAR test file hash

}

def check_ioc(ioc_type, ioc_value):

    endpoint_map = {"ips": "ip_addresses", "domains": "domains", "hashes": "files"}

    url = f"{BASE_URL}/{endpoint_map[ioc_type]}/{ioc_value}"

    try:

        response = requests.get(url, headers=HEADERS, timeout=10)

        if response.status_code == 200:

            data = response.json()["data"]["attributes"]

            stats = data.get("last_analysis_stats", {})

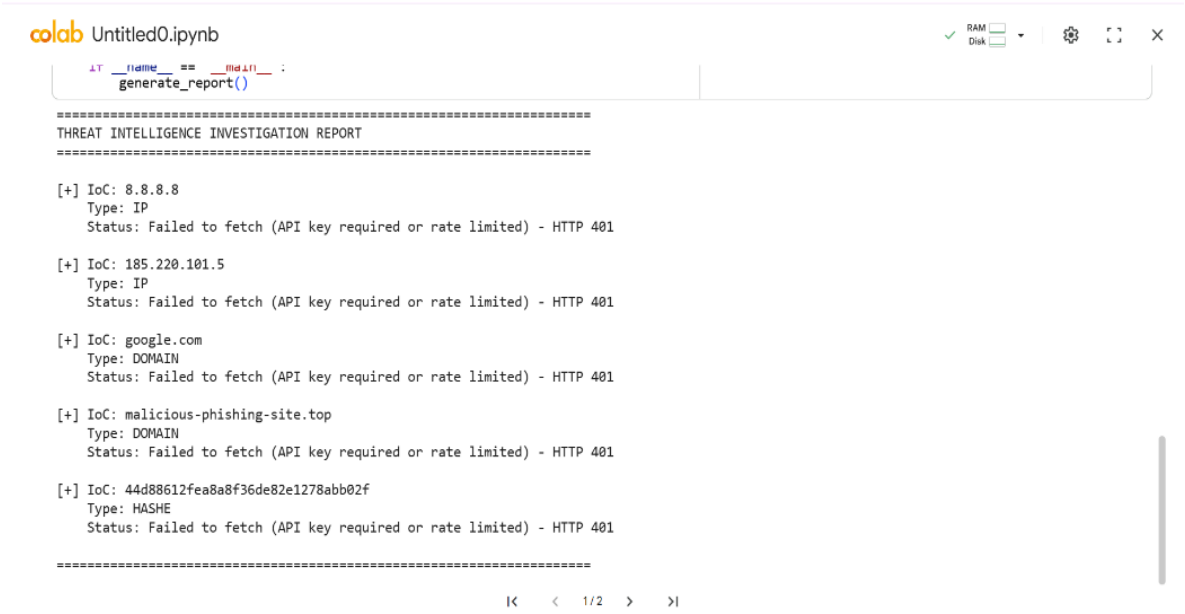
            malicious = stats.get("malicious", 0)

            suspicious = stats.get("suspicious", 0)
```

```
            harmless = stats.get("harmless", 0)
            undetected = stats.get("undetected", 0)
            # Determine risk classification
            if malicious > 5:
                verdict = "HIGH RISK / MALICIOUS"
                action = "Block entity across firewall/EDR immediately and isolate affected hosts."
            elif malicious > 0 or suspicious > 2:
                verdict = "MEDIUM RISK / SUSPICIOUS"
                action = "Flag for active monitoring and restrict network permissions."
            else:
                verdict = "LOW RISK / CLEAN"
                action = "No immediate action required."
            return {
                "IoC": ioc_value,
                "Type": ioc_type.upper()[:-1],
                "Reputation Score": f"{malicious}/{malicious + suspicious + harmless + undetected} detections",
                "Verdict": verdict,
                "Recommended Action": action,
            }
        else:
            return {
                "IoC": ioc_value,
                "Type": ioc_type.upper()[:-1],
                "Error": f"HTTP {response.status_code}",
            }
```

```
    except Exception as e:
        return {"IoC": ioc_value, "Type": ioc_type.upper()[:-1], "Error": str(e)}
def generate_report():
    print("=" * 70)
    print("THREAT INTELLIGENCE INVESTIGATION REPORT")
    print("=" * 70)
    for ioc_type, items in IOCS.items():
        for item in items:
            result = check_ioc(ioc_type, item)
            print(f"\n[+] IoC: {result.get('IoC')}")
            print(f"    Type: {result.get('Type')}")
            if "Error" in result:
                print(
                    f"    Status: Failed to fetch (API key required or rate limited) - {result['Error']}"
                )
            else:
                print(f"    Detections: {result.get('Reputation Score')}")
                print(f"    Verdict: {result.get('Verdict')}")
                print(f"    Action: {result.get('Recommended Action')}")
    print("\n" + "=" * 70)
if __name__ == "__main__":
    generate_report()
```

**Output**




## Exercise 2: Security Log Collection and Correlation

This script ingests sample logs from three sources (Authentication, Firewall, and Web Server), correlates them using target IP addresses and user sessions, and highlights suspicious cross-system attack patterns.

Python

```
from collections import defaultdict

import datetime

# Mock Auth Logs (SSH/Login attempts)

auth_logs = [

    {

        "timestamp": "2026-10-30 08:00:12",

        "user": "admin",

        "src_ip": "203.0.113.45",

        "event": "FAILED_LOGIN",

    },

    {

        "timestamp": "2026-10-30 08:00:15",
```

```
        "user": "admin",
        "src_ip": "203.0.113.45",
        "event": "FAILED_LOGIN",
    },
    {
        "timestamp": "2026-10-30 08:00:20",
        "user": "admin",
        "src_ip": "203.0.113.45",
        "event": "SUCCESSFUL_LOGIN",
    },
    {
        "timestamp": "2026-10-30 08:05:00",
        "user": "john",
        "src_ip": "192.168.1.10",
        "event": "SUCCESSFUL_LOGIN",
    },
]
# Mock Firewall Logs
firewall_logs = [
    {
        "timestamp": "2026-10-30 08:00:05",
        "src_ip": "203.0.113.45",
        "dst_port": 22,
        "action": "ALLOW",
    },
```

```
    {
        "timestamp": "2026-10-30 08:01:00",
        "src_ip": "203.0.113.45",
        "dst_port": 80,
        "action": "ALLOW",
    },
    {
        "timestamp": "2026-10-30 08:02:10",
        "src_ip": "198.51.100.7",
        "dst_port": 443,
        "action": "DENY",
    },
]
# Mock Web Server Logs (HTTP requests)
web_logs = [
    {
        "timestamp": "2026-10-30 08:01:05",
        "src_ip": "203.0.113.45",
        "uri": "/admin/config.php",
        "status": 200,
    },
    {
        "timestamp": "2026-10-30 08:01:10",
        "src_ip": "203.0.113.45",
        "uri": "/etc/passwd",
```

```
        "status": 403,
    },
    {
        "timestamp": "2026-10-30 08:05:10",
        "src_ip": "192.168.1.10",
        "uri": "/dashboard",
        "status": 200,
    },
]
def correlate_events():
    correlated_data = defaultdict(
        lambda: {
            "failed_logins": 0,
            "successful_logins": 0,
            "web_requests": [],
            "firewall_actions": [],
        }
    )
    # Correlate Auth
    for log in auth_logs:
        ip = log["src_ip"]
        if log["event"] == "FAILED_LOGIN":
            correlated_data[ip]["failed_logins"] += 1
        elif log["event"] == "SUCCESSFUL_LOGIN":
            correlated_data[ip]["successful_logins"] += 1
```

```
    # Correlate Firewall
    for log in firewall_logs:
        ip = log["src_ip"]
        correlated_data[ip]["firewall_actions"].append(
            f"{log['action']} port {log['dst_port']}"
        )
    # Correlate Web
    for log in web_logs:
        ip = log["src_ip"]
        correlated_data[ip]["web_requests"].append(
            f"{log['uri']} (HTTP {log['status']})"
        )
    print("=" * 70)
    print("CORRELATED SECURITY EVENT SUMMARY")
    print("=" * 70)
    for ip, activity in correlated_data.items():
        print(f"\n[+] Correlated Profile for Source IP: {ip}")
        print(f"  - Auth Attempts : {activity['failed_logins']} Failed, {activity['successful_logins']} Successful")
        print(f"  - Firewall Hits : {', '.join(activity['firewall_actions']) if activity['firewall_actions'] else 'None'}")
        print(f"  - Web Activity  : {', '.join(activity['web_requests']) if activity['web_requests'] else 'None'}")
        # Correlation logic rule: Brute force followed by admin endpoint traversal
        if activity["failed_logins"] >= 2 and activity["web_requests"]:
            print(
```

```
            "    [!] ALERT: Potential Compromise Pattern Detected (Brute Force -> Web Probing)"
        )

if __name__ == "__main__":
    correlate_events()
```

**Output**




## Exercise 3: Threat Hunting Using Network Logs

This script processes network traffic logs to detect anomalies such as beaconing activity, unexpected high-frequency outbound connections, and unapproved external destinations.

Python

```
from collections import Counter

import pandas as pd

# Sample Network Traffic Dataset

network_traffic = [

    {

        "timestamp": "2026-10-30 09:00:01",
```

```
    "src_ip": "10.0.0.15",
    "dst_ip": "198.51.100.99",
    "dst_port": 443,
    "bytes_sent": 512,
},
{
    "timestamp": "2026-10-30 09:00:31",
    "src_ip": "10.0.0.15",
    "dst_ip": "198.51.100.99",
    "dst_port": 443,
    "bytes_sent": 512,
},
{
    "timestamp": "2026-10-30 09:01:01",
    "src_ip": "10.0.0.15",
    "dst_ip": "198.51.100.99",
    "dst_port": 443,
    "bytes_sent": 512,
},
{
    "timestamp": "2026-10-30 09:01:31",
    "src_ip": "10.0.0.15",
    "dst_ip": "198.51.100.99",
    "dst_port": 443,
    "bytes_sent": 512,
```

```
    },
    {
        "timestamp": "2026-10-30 09:00:10",
        "src_ip": "10.0.0.22",
        "dst_ip": "142.250.190.46",
        "dst_port": 80,
        "bytes_sent": 1200,
    },
    {
        "timestamp": "2026-10-30 09:02:15",
        "src_ip": "10.0.0.105",
        "dst_ip": "203.0.113.200",
        "dst_port": 8080,
        "bytes_sent": 1048576,
    },
]
def perform_threat_hunt():
    df = pd.DataFrame(network_traffic)
    print("=" * 70)
    print("THREAT HUNTING REPORT: NETWORK TRAFFIC ANALYSIS")
    print("=" * 70)
    # 1. Identify High-Frequency Repeated Connections (Potential Beaconing)
    connection_counts = (
        df.groupby(["src_ip", "dst_ip", "dst_port"])
        .size()
```

```
        .reset_index(name="connection_count")
    )
    beaconing_candidates = connection_counts[
        connection_counts["connection_count"] >= 3
    ]
    print("\n[1] SUSPICIOUS BEACONING PATTERNS (Repeated Connections)")
    if not beaconing_candidates.empty:
        for _, row in beaconing_candidates.iterrows():
            print(f"    - Source IP      : {row['src_ip']}")
            print(f"      Destination IP : {row['dst_ip']}:{row['dst_port']}")
            print(f"      Frequency      : {row['connection_count']} attempts")
            print(
                "      Finding        : Regular time interval output detected. High probability of C2 beaconing."
            )
    else:
        print("    No beaconing patterns detected.")
    # 2. Identify Large Outbound Data Transfers (Potential Data Exfiltration)
    data_transfers = (
        df.groupby(["src_ip", "dst_ip"])["bytes_sent"]
        .sum()
        .reset_index(name="total_bytes")
    )
    large_transfers = data_transfers[data_transfers["total_bytes"] > 500000]
    print("\n[2] ANOMALOUS HIGH-VOLUME DATA EXFILTRATION")
    if not large_transfers.empty:
```

```
        for _, row in large_transfers.iterrows():
            print(f"    - Source IP      : {row['src_ip']}")
            print(f"      Destination IP : {row['dst_ip']}")
            print(f"      Volume Transferred: {row['total_bytes']} bytes")
            print(
                "      Finding        : Abnormally large outbound transfer on non-standard port."
            )
    else:
        print("    No anomalous outbound data transfers detected.")
    # 3. Summary Recommendations
    print("\n" + "-" * 70)
    print("RECOMMENDED MITIGATION ACTIONS:")
    print(" 1. Block destination IP 198.51.100.99 on outbound firewall.")
    print(
        " 2. Isolate internal host 10.0.0.15 for forensic memory analysis."
    )
    print(
        " 3. Investigate process list on 10.0.0.105 for unauthorized archive/exfiltration tools."
    )
    print("=" * 70)
if __name__ == "__main__":
    perform_threat_hunt()
```

**Output**

```
        print("=" * 70)


    if __name__ == "__main__":
        perform_threat_hunt()
```

```
======================================================================
THREAT HUNTING REPORT: NETWORK TRAFFIC ANALYSIS
======================================================================

[1] SUSPICIOUS BEACONING PATTERNS (Repeated Connections)
    - Source IP      : 10.0.0.15
      Destination IP : 198.51.100.99:443
      Frequency      : 4 attempts
      Finding        : Regular time interval output detected. High probability of C2 beaconing.

[2] ANOMALOUS HIGH-VOLUME DATA EXFILTRATION
    - Source IP      : 10.0.0.105
      Destination IP : 203.0.113.200
      Volume Transferred: 1048576 bytes
      Finding        : Abnormally large outbound transfer on non-standard port.

----------------------------------------------------------------------
RECOMMENDED MITIGATION ACTIONS:
 1. Block destination IP 198.51.100.99 on outbound firewall.
 2. Isolate internal host 10.0.0.15 for forensic memory analysis.
 3. Investigate process list on 10.0.0.105 for unauthorized archive/exfiltration tools.
======================================================================
```